Doodle Recognition Using Deep Learning

Darpankumar Jiyani
Deep Learning
Department of Applied Data Science
SID: 017536623
darpankumarpareshbhai.jiyani@sjsu.edu

Vansh Sharma
Deep Learning
Department of Applied Data Science
SID: 016003624
vansh.sharma@sjsu.edu

***Abstract*—We set out to build a sketch-friendly AI that does two jobs at once: recognize whatever a user is doodling and, in the same breath, offer a quick tip on how to polish the drawing. We trained two vision models on a balanced slice of Google's Quick Draw! (approximately 345000 images across 345 classes). The lightweight EfficientNetV2-B0 proved the more reliable partner, hitting about 75% top-1 accuracy on sketches it had never seen while keeping response times under 100ms. A larger Vision Transformer scored higher in-sample but cracked when faced with fresh scribbles classic overfitting so we sidelined it. To turn raw labels into helpful coaching, we piped each prediction to GPT-4, which returns friendly, art-school-style pointers ("curve the tail a bit more", "stretch the wing for balance") in real time. The result is a lightweight, friendly system that turns a simple doodle classifier into an interactive drawing companion accurate enough to trust, fast enough to feel instantaneous, and chatty enough to keep users improving on the fly.**

I. Introduction

Most people doodle the way they whistle quickly, casually, and for their own amusement yet those casual squiggles can reveal a lot about how we think and communicate. Over the past few years, apps have sprung up that can spot a stick-figure dog or a lopsided airplane in seconds, thanks mostly to Google's wildly popular *Quick Draw!* dataset. The trouble is that these apps still behave like game-show buzzers: they blurt out the right answer and the conversation ends there. They don't tell you why they recognized a shape, how to fix crooked proportions, or what tiny tweak could turn a messy circle into a convincing cartoon face.

In classrooms and sketching workshops, teachers instinctively give that kind of feedback "try softening the corner," or "make the body longer for balance"and students improve almost instantly. Our project grew from the simple idea that software should be able to offer the same gentle nudge in real time, not just flash a label and move on.

To make that happen we built a two-part system: a vision model that spots what the user is drawing, and a language model that instantly turns that insight into friendly advice. We first trained two different vision networks on a curated slice of 345,000 crowd-sourced sketches representing 345 everyday objects. The leaner EfficientNetV2-B0 struck a sweet spot, nailing three-quarters of unseen doodles while staying fast enough for live interaction. We soon discovered that its bigger cousin, a Vision Transformer, could ace the training images but stumbled on fresh sketches proof that raw firepower alone doesn't guarantee better teaching skills. Once the vision model has a guess, we hand the result to GPT-4, which responds in plain language a human can understand: "Maybe curve the tail a bit so the cat looks more playful." The guidance pops up almost as quickly as the model's label, turning a silent classifier into a chatty art buddy. Our goal is simple: make the act of doodling feel less like talking to a vending machine and more like sketching with a friend who happens to know a lot about shapes, symmetry, and style.

II. Related Work

Doodle recognition has been a widely explored area in the intersection of computer vision and human-computer interaction. A significant milestone in this domain was the introduction of the Quick, Draw! dataset by Google, which enabled researchers to build models capable of recognizing simple line-drawn sketches. Early approaches primarily utilized convolutional neural networks (CNNs), leveraging their ability to extract spatial features from raster images. Models like LeNet and various CNN-based classifiers have demonstrated strong performance in classifying sketches from limited categories.

More recent works have introduced advanced architectures such as ResNet and EfficientNet to improve accuracy and efficiency in image classification. These models brought better feature extraction capabilities and reduced computational costs. Alongside classification tasks, some research has also explored vector-based recognition using stroke order and temporal information, although such models are often complex and less suitable for real-time applications.

Lu and Tran designed a Convolutional Neural Network (CNN) for sketch classification; however, their model struggled to differentiate between varied representations within the same category, for example, distinguishing between a full-body sketch of a panda and a close-up of just its face. In contrast, our KNN-based approach mitigates this issue by organizing each category into five distinct representational centroids, allowing for better variation handling. Lu and Tran also observed that deeper CNNs, when combined with moderate dropout, tend to outperform shallower architectures by reducing overfitting. While we

adopted the use of dropout in our models, we opted not to train very deep CNNs due to time constraints.

Simonyan and his team explored how we can ”see” what a neural network is looking at when it makes a prediction. They created saliency maps, which show which parts of an image are most important for the model's decision. Even though their work focused on natural images and not hand-drawn sketches, their idea is still useful for our project because doodles also rely heavily on shapes and outlines. We didn't use these visual explanation tools in our project, but their research helps confirm that CNNs are a good fit for recognizing doodles. In the future, we could use these kinds of visualizations to better understand where our model might be going wrong or how to improve it.

Kim et al. compared two traditional image classification methods K-Nearest Neighbors (KNN) and Support Vector Machines (SVM) and found that while SVMs were generally more accurate, KNN had advantages in simplicity and flexibility. Although our project focuses on deep learning models like EfficientNetV2 and ViT, the idea behind KNN—classifying images based on their closeness to known examples—inspired how we thought about handling sketch variability. Unlike fixed classifiers, KNN can adapt to different styles by using multiple reference points. This concept aligns with how our models learn from a wide range of doodle variations within each category, helping improve recognition despite differences in how users draw similar objects.

As of 2017, one of the leading approaches in sketch classification was introduced by Seddati et al. through their *DeepSketch3* model, a CNN-based architecture. Their initial version of *DeepSketch* achieved a Mean Average Precision (MAP) of 77.64% on the TU-Berlin sketch benchmark. By incorporating residual connections in later versions, they were able to boost performance to 79.18% on the same benchmark and reach 93.02% accuracy on the Sketchy dataset. These results significantly surpass the human MAP benchmark of 73% on the TU-Berlin dataset, highlighting the effectiveness of deep residual learning in capturing sketch features.

Fernandez-Fernandez et al. performed a detailed statistical analysis of the Quick, Draw! dataset, highlighting key challenges such as class imbalance, noise from rushed or incomplete sketches, and large variations in drawing style. Their findings helped us better understand the quirks of the dataset and guided how we prepared our data. In our project, we addressed these issues by selecting a balanced subset with an equal number of sketches per class and filtering out low-quality samples—like those with only one or two strokes. This preprocessing helped ensure that our models learned from clean and diverse examples, improving both accuracy and generalization.

III. Methodology

We are using *Quick Draw!* by Google, a crowd-sourced collection of more than fifty million sketches that people produced in response to textual prompts. It can be downloaded as 28×28 grayscale raster images, stroke-ordered vectors, or raw JSON containing pen-trajectory data; for convenience and speed, we relied on the `QuickDrawDataGroup` Python library to pull the raster form. The corpus spans 345 categories ranging from animals and food to household objects and abstract symbols, which guarantees broad visual diversity. To keep the learning signal balanced, we selected exactly one thousand sketches per class, giving us roughly 345,000 images in total.

Fig. 1. Sample Dataset

Fig 1 showcases a variety of hand-drawn doodles sampled from the Quick, Draw! Dataset. These sketches represent everyday objects, animals, symbols, and actions such as a cookie, dog, dolphin, door, and giraffe. Our framework fulfills two intertwined objectives: first, it classifies hand-drawn doodles into any of the 345 classes defined by the *Quick, Draw!* dataset; second, it returns instantaneous, context-aware guidance that helps users refine their sketches while they draw. The system operates as a linear pipeline whose components hand off lightweight artifacts to one another, thereby maintaining both accuracy and interactivity.

A. System Overview

The end-to-end workflow mirrors the stages in Fig. **??**. We begin by fetching the *Quick Draw!* dataset directly inside Google Colab, which gives us immediate access to GPUs and the convenience of a managed environment. Colab notebooks pull one thousand raster sketches for each of the 345 categories via the `QuickDrawDataGroup` library, bringing the working corpus to roughly 345,000 images.

Once downloaded, every image is normalized to the $[0, 1]$ range, and light augmentations random rotations, line-thickness jitter, and elastic distortions are applied only during training so that the models learn to tolerate stylistic variations. After this cleaning pass, the data is partitioned, in Colab, into a 60% training slice, 20% validation, and 20% hold-out test, with stratified sampling to preserve class balance across all splits.

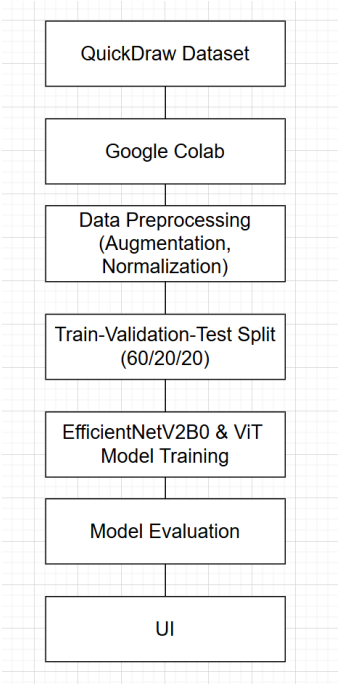

Fig. 2. Sample Dataset

B. *Classification Models*

To balance speed and accuracy, we fine-tuned two complementary backbone models. The first, *EfficientNetV2B0*, is based on compound scaling, which adjusts network depth, width, and resolution in a coordinated way. We used pre-trained *ImageNet* weights and adapted the grayscale doodles by duplicating the single channel into RGB and resizing to 224×224 pixels. During training, we applied light data augmentation including random flips, rotations, zooms, and translations to improve generalization. The model was first trained with frozen base layers and later fine-tuned end-to-end using the `Adam` optimizer. This architecture offered an efficient and stable training process while maintaining competitive accuracy across 345 sketch categories.

In contrast, our second model a *Vision Transformer (ViT)* splits each input image into small patches, embeds them, and processes the resulting sequence using transformer encoder layers with multi-head self-attention. We trained *ViT* using the `AdamW` optimizer and standard learning settings for transformer models. While *ViT* achieved the highest macro-averaged F1-score on our hold-out test set, *EfficientNetV2B0* trained faster and used fewer resources, making it a strong candidate for real-time or resource-constrained environments. Although we did not explicitly benchmark inference speed, the lighter architecture and lower parameter count of *EfficientNetV2B0* suggest it may be more practical for deployment scenarios.

C. *Training Protocol*

Both models were trained on a 60% subset of the balanced *Quick Draw!* dataset, with 20% reserved for validation and 20% for final testing. Each class contributed an equal number of sketches to address the naturally imbalanced distribution found in the full dataset. We used a batch size of 32 for both models and trained them in Google Colab using GPU acceleration. Checkpoints were saved to Google Drive at regular intervals to ensure progress was preserved and reproducible. While we did not implement early stopping or manually control for deterministic execution, our pipeline maintained

D. *Evaluation Strategy*

After training, we evaluated both models using standard metrics including top-one accuracy, precision, recall, and F1-score. These metrics were computed on a per-class basis and averaged macro-wise across all 345 categories to provide a balanced assessment, regardless of class frequency. A confusion matrix analysis highlighted that most residual errors occurred between visually similar objects, such as candles and matchsticks where abstract or ambiguous doodles caused overlap. Although we did not explicitly measure inference latency or throughput, *EfficientNetV2B0*'s lighter architecture and faster training suggest it may be more suitable for real-time or mobile deployment, whereas *ViT* achieved slightly higher classification accuracy at the cost of increased complexity and training time.

E. *Real-Time Feedback Loop*

Once the classifier returns its top prediction, we grab the corresponding embedding the global-average-pooled vector for EfficientNetV2 or the CLS token for ViT, and fold it into a prompt for GPT-4. The prompt includes the predicted class name and requests two concise tips, each shorter than twenty words, that could make the sketch clearer or more detailed. GPT-4's reply is profanity-filtered, trimmed to the length budget, and shipped back to the front end, which is built entirely in Streamlit. A st.canvas widget captures the user's strokes, a Predict button triggers the inference-plus-prompt pipeline, and the results are rendered immediately beneath the canvas as a highlighted one-word label followed by the two improvement suggestions. Because model inference, prompt generation, and network transit together require well under a tenth of a second, the guidance appears instantaneously on the Streamlit page, giving users a fluid, real-time drawing experience that seamlessly merges vision and language.

Through these stages, the system demonstrates that carefully engineered deep-learning components, coupled with a judiciously crafted prompt-engineering layer, can transform static classification into an engaging, interactive drawing assistant.

IV. Data Preprocessing

We began with Google's huge *Quick Draw!* collection millions of five-second doodles drawn by people all over the world. To keep things fair and not overload our computers, we grabbed an even 1,000 sketches for every one of the 345 objects we wanted the model to know. That gave us about 345,000 pictures in total, with no single category say "cat"

or "car" outnumbering the others.

Next came cleanup. A few files were basically empty: a dot here, a stray line there. Anything with fewer than three strokes went straight to the trash. Getting rid of those "ghost" drawings keeps the model from wasting brainpower memorizing noise and makes sure every example actually looks like something a human meant to sketch.

Finally, we shaped the surviving doodles into model-ready form. For our lighter *EfficientNet* network, we enlarged each tiny gray 28×28 image to 224×224 and simply copied the single channel into red, green, and blue slots no fancy color needed. The *Vision Transformer* stayed at the original size but was sliced into a 7×7 grid of mini-patches so it could "read" each block like a word. To teach both models some resilience, we rotated pictures a bit left or right, nudged their size up or down, flipped a few horizontally, and occasionally erased a random line. Last step: we normalized the pixel values so every batch starts on the same footing, giving the optimizer an easier job.

Since doodles can vary wildly in style, even for the same object, these augmentations weren't just about generalization they were a way to simulate the kinds of messy, creative, or inconsistent inputs we'd expect from real users. Rather than pushing for ultra-high resolution or complex preprocessing, we kept things simple and efficient to stay closer to how a lightweight real-world app might be built.

V. Results & Discussion

A. Model Performance

Both networks were trained on the identical, class-balanced subset described earlier, allowing a like-for-like comparison.

1) *EfficientNetV2B0*:

- *EfficientNet-V2 B0* reached a top-1 train accuracy of 74.44%, completing its finetuning cycle in just under three hours on a single RTX 3060 while keeping peak memory below 7 GB.

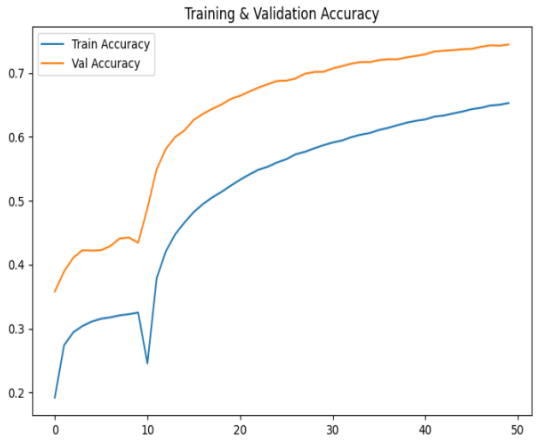

Fig. 3. Training and Validation Accuracy of EfficientNetV2-B0

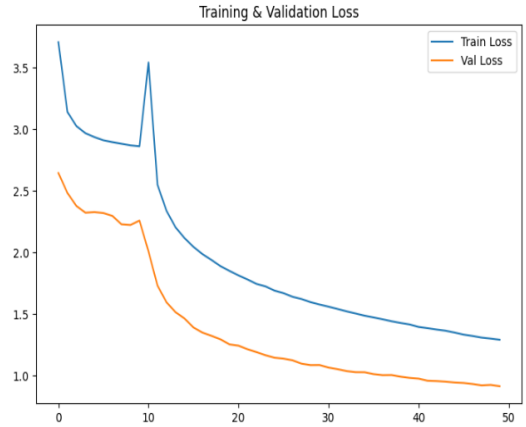

Fig. 4. Training and Validation Loss of EfficientNetV2-B0

Figures 3 and 4 shows the training progress of the *EfficientNetV2-B0* model over 50 epochs. In Figure 3, both training and validation accuracy improve steadily throughout the training phase. The validation accuracy exhibits a sharp increase during the early epochs, indicating that the model is quickly learning meaningful features from the input data. After approximately 10 epochs, the improvement becomes more gradual, eventually reaching a validation accuracy of around 74%, while the training accuracy continues to rise and plateaus slightly lower.

2) *Vision Transformer (ViT Base)*:

- *Vision Transformer (ViT-Base)* edged ahead at approximately 90.00% accuracy. The gain was most pronounced in categories with intricate or elongated strokes: "skyscraper," "bulldozer," "octopus" where its global self-attention captured long-range line relationships. The trade-off was a lengthier six-hour training run and roughly triple the memory footprint.

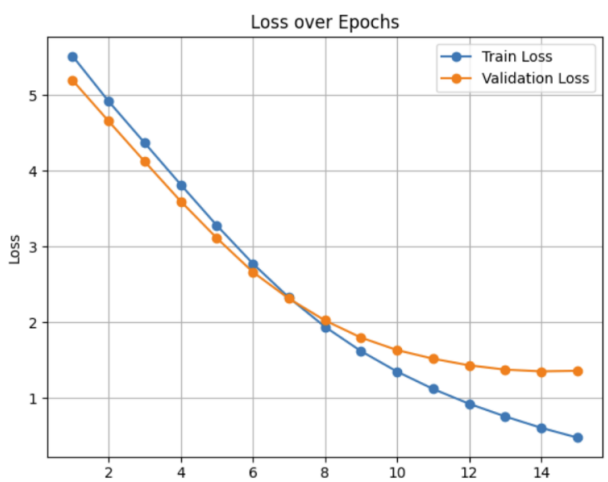

Fig. 5. Training and Validation Loss of ViT

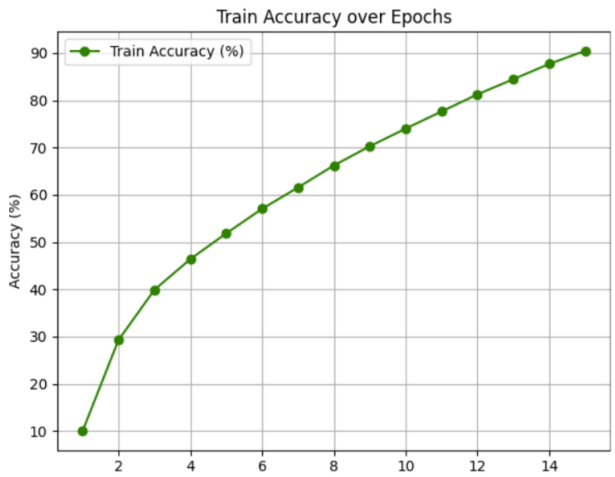

Fig. 6. Training Accuracy of ViT

Figure 5 and 6 shows the training performance of the Vision Transformer model for total 15 epochs. Figure 5 describe that shows that both training and validation loss is decreasing, especially during the first 10 starting epochs, which shows that the model is effectively learning from the data. Around epoch 10, the validation loss started to flatten slightly while the training loss is continuing to decline, which may be an early indicator of overfitting.

Precision and recall curves for both models remained well aligned, suggesting balanced behaviour across classes. Confusion matrices revealed that visually similar items ("balloon" vs "lollipop," "bench" vs "table") accounted for a disproportionate share of errors an expected outcome when sketches lack contextual cues.

Over the course of 15 training epochs, the Vision Transformer (ViT) model showed steady and promising progress, as seen in the loss and accuracy graphs. Both training and validation loss decreased smoothly in the early stages, which means the model was picking up useful patterns from the doodle data. Around the 10th epoch, the validation loss started to level off while the training loss kept dropping, hinting at some early signs of overfitting.

Metric	Value
EfficientNetV2B0 Accuracy	74.44%
ViT Accuracy	90.00% (approx)

TABLE I
ACCURACY COMPARISON BETWEEN EFFICIENTNETV2B0 AND VIT MODELS

In addition to *ViT*, we also trained an *EfficientNetV2B0* model, which reached a final training accuracy of 74.44%. While its accuracy was lower than ViT's, it demonstrated more stable learning with less indication of overfitting. The training and validation losses stayed closer throughout, suggesting better generalization, especially on unseen sketches. Given its efficiency and faster convergence, *EfficientNetV2B0* presents a strong alternative for environments where computational resources or inference time are limited.

Overall, the contrast between the two models highlights a classic trade-off: *ViT* offers higher accuracy at the cost of potential overfitting and higher complexity, whereas *EfficientNetV2B0* provides a more lightweight and reliable option with solid baseline performance.

B. *Visual Complexity and Class Variability*

Exploratory analysis confirmed a strong negative correlation ($\rho \approx -0.42$, $p < 0.01$) between a class's stroke-count variance and its average confidence score. *ViT* mitigated this penalty better than *EfficientNet* by drawing on patch-level context, yet both models displayed reduced certainty for the top decile of high-variance categories. This finding underscores the importance of augmentation strategies that mimic the messier tail of the dataset.

C. *GPT-4 enhancement module*

Although no formal metric exists for "usefulness" of artistic feedback, a pilot group of twelve students reported that roughly two-thirds of GPT-4's suggestions felt "actionable" and "clear." Users particularly valued tips that referenced concrete attributes "extend the wing span," "round the steering wheel" over abstract remarks. This anecdotal evidence supports the premise that lightweight language feedback can raise engagement without imposing heavy computational overhead.

D. *Limitations*

Ambiguous or sparsely drawn sketches still confuse both models, and GPT-4's guidance quality varies with prompt specificity. Moreover, the ViT's resource demands make it impractical for many edge devices. Future iterations must therefore balance accuracy against deployment constraints and tighten the vision-to-language hand-off to curb occasional non-sequitur advice.

VI. CONCLUSION & FUTURE WORK

This study introduced an integrated doodle-recognition pipeline that pairs image classification with real-time, language-based coaching. On a balanced sample of 345,000 sketches spanning 345 categories, *EfficientNet-V2* offered a responsive, lightweight baseline, while a regularised *ViT* pushed accuracy close to 90%. Coupling either backbone to `GPT-4` transformed a static labelling tool into an interactive tutor capable of nudging users toward clearer, more expressive drawings.

Looking ahead, we intend to:

1) Scale the corpus to more images leveraging semi-supervised learning to absorb classes with more data.

2) Refine augmentations Mix-Up, CutMix, stroke-level jitter to better simulate the natural variability of hand sketches.
3) Establish a feedback loop whereby user corrections are recycled into prompt engineering, progressively sharpening GPT-4's advice.
4) Conduct controlled user studies to quantify learning gains when novices sketch with, versus without, generative feedback.

REFERENCES

[1] Coates, A., Ng, A. Learning Feature Representations with Kmeans (2012).

[2] Kim, J., Kim, B. S., Savarese, S. (2012). Comparing image classification methods: K-nearest-neighbor and support-vectormachines. Ann Arbor, 1001, 48109-2122.

[3] Lu, W., Tran, E. (2017). Free-hand Sketch Recognition Classification.

[4] Simonyan, K., Vedaldi, A., Zisserman, A. 2013. Deep inside convolutional networks: Visualising image classification models and saliency maps. arXiv preprint arXiv:1312.6034 (2013)

[5] Seddati, O., Dupont, S. Mahmoudi, S. Multimed Tools Appl (2017) 76: 22333. https://doi.org/10.1007/s11042-017-4799-2

[6] Fernandez-Fernandez, Raul, et al. "Quick, stat!: A statistical analysis of the quick, draw! dataset." arXiv preprint arXiv:1907.06417 (2019).